In [211…

```
# This cell is added by sphinx-gallery
%matplotlib inline
%pip install pennylane torch torchvision
```

```
Requirement already satisfied: pennylane in /home/msoliviawatt/machine_learn
ing/ml-env/lib/python3.13/site-packages (0.44.1)
Requirement already satisfied: torch in /home/msoliviawatt/machine_learning/
ml-env/lib/python3.13/site-packages (2.11.0)
Requirement already satisfied: torchvision in /home/msoliviawatt/machine_lea
rning/ml-env/lib/python3.13/site-packages (0.26.0)
Requirement already satisfied: scipy in /home/msoliviawatt/machine_learning/
ml-env/lib/python3.13/site-packages (from pennylane) (1.17.1)
Requirement already satisfied: networkx in /home/msoliviawatt/machine_learni
ng/ml-env/lib/python3.13/site-packages (from pennylane) (3.6.1)
Requirement already satisfied: rustworkx>=0.14.0 in /home/msoliviawatt/machi
ne_learning/ml-env/lib/python3.13/site-packages (from pennylane) (0.17.1)
Requirement already satisfied: autograd in /home/msoliviawatt/machine_learni
ng/ml-env/lib/python3.13/site-packages (from pennylane) (1.8.0)
Requirement already satisfied: appdirs in /home/msoliviawatt/machine_learnin
g/ml-env/lib/python3.13/site-packages (from pennylane) (1.4.4)
Requirement already satisfied: autoray==0.8.2 in /home/msoliviawatt/machine_
learning/ml-env/lib/python3.13/site-packages (from pennylane) (0.8.2)
Requirement already satisfied: cachetools in /home/msoliviawatt/machine_lear
ning/ml-env/lib/python3.13/site-packages (from pennylane) (7.0.5)
Requirement already satisfied: pennylane-lightning>=0.44 in /home/msoliviawa
tt/machine_learning/ml-env/lib/python3.13/site-packages (from pennylane) (0.
44.0)
Requirement already satisfied: requests in /home/msoliviawatt/machine_learni
ng/ml-env/lib/python3.13/site-packages (from pennylane) (2.32.5)
Requirement already satisfied: tomlkit in /home/msoliviawatt/machine_learnin
g/ml-env/lib/python3.13/site-packages (from pennylane) (0.14.0)
Requirement already satisfied: typing_extensions in /home/msoliviawatt/machi
ne_learning/ml-env/lib/python3.13/site-packages (from pennylane) (4.15.0)
Requirement already satisfied: packaging in /home/msoliviawatt/machine_learn
ing/ml-env/lib/python3.13/site-packages (from pennylane) (26.0)
Requirement already satisfied: diastatic-malt==2.15.2 in /home/msoliviawatt/
machine_learning/ml-env/lib/python3.13/site-packages (from pennylane) (2.15.
2)
Requirement already satisfied: gast in /home/msoliviawatt/machine_learning/m
l-env/lib/python3.13/site-packages (from pennylane) (0.7.0)
Requirement already satisfied: numpy in /home/msoliviawatt/machine_learning/
ml-env/lib/python3.13/site-packages (from pennylane) (2.4.2)
Requirement already satisfied: astunparse in /home/msoliviawatt/machine_lear
ning/ml-env/lib/python3.13/site-packages (from diastatic-malt==2.15.2->penny
lane) (1.6.3)
Requirement already satisfied: termcolor in /home/msoliviawatt/machine_learn
ing/ml-env/lib/python3.13/site-packages (from diastatic-malt==2.15.2->pennyl
ane) (3.3.0)
Requirement already satisfied: filelock in /home/msoliviawatt/machine_learni
ng/ml-env/lib/python3.13/site-packages (from torch) (3.25.2)
Requirement already satisfied: setuptools<82 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from torch) (81.0.0)
Requirement already satisfied: sympy>=1.13.3 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from torch) (1.14.0)
Requirement already satisfied: jinja2 in /home/msoliviawatt/machine_learnin
g/ml-env/lib/python3.13/site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from torch) (2026.3.0)
Requirement already satisfied: cuda-toolkit==13.0.2 in /home/msoliviawatt/ma
chine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit[cubla
```

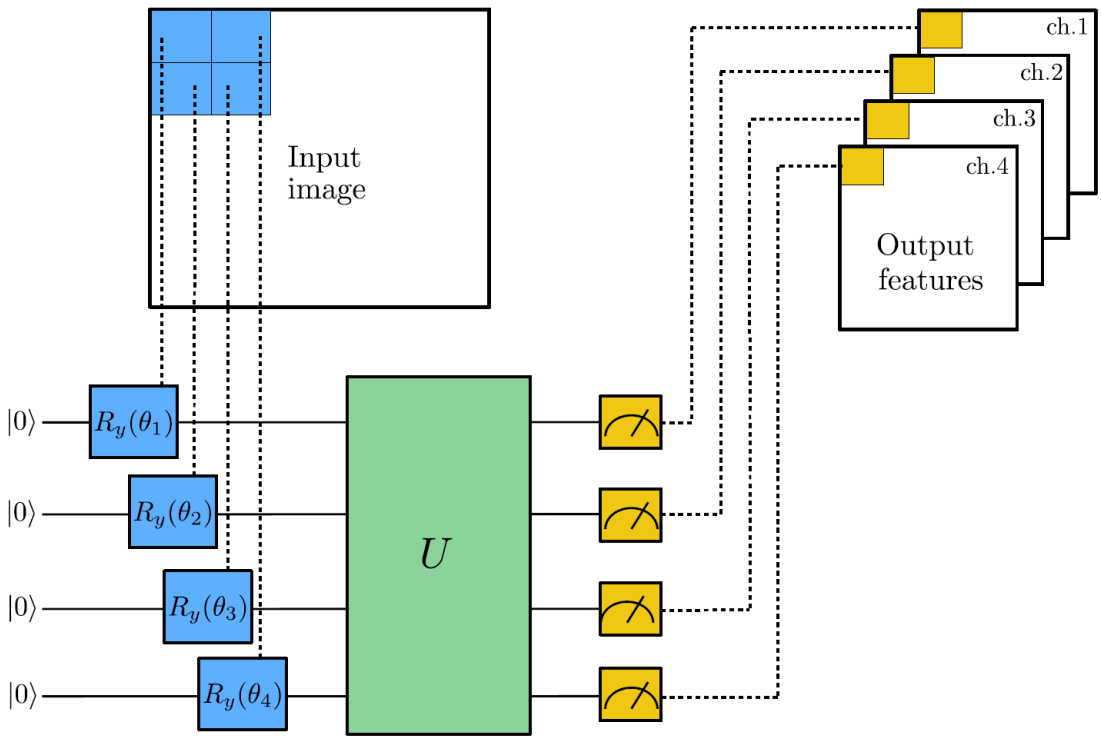
```
s,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==
13.0.2; platform_system == "Linux"->torch) (13.0.2)
Requirement already satisfied: cuda-bindings<14,>=13.0.3 in /home/msoliviawa
tt/machine_learning/ml-env/lib/python3.13/site-packages (from torch) (13.2.
0)
Requirement already satisfied: nvidia-cudnn-cu13==9.19.0.56 in /home/msolivi
awatt/machine_learning/ml-env/lib/python3.13/site-packages (from torch) (9.1
9.0.56)
Requirement already satisfied: nvidia-cusparselt-cu13==0.8.0 in /home/msoliv
iawatt/machine_learning/ml-env/lib/python3.13/site-packages (from torch) (0.
8.0)
Requirement already satisfied: nvidia-nccl-cu13==2.28.9 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from torch) (2.28.9)
Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in /home/msoliviaw
att/machine_learning/ml-env/lib/python3.13/site-packages (from torch) (3.4.
5)
Requirement already satisfied: triton==3.6.0 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from torch) (3.6.0)
Requirement already satisfied: nvidia-cublas==13.1.0.3.* in /home/msoliviawa
tt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit[c
ublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvt
x]==13.0.2; platform_system == "Linux"->torch) (13.1.0.3)
Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in /home/msoli
viawatt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-tool
kit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrt
c,nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.96)
Requirement already satisfied: nvidia-cufft==12.0.0.61.* in /home/msoliviawa
tt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit[c
ublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvt
x]==13.0.2; platform_system == "Linux"->torch) (12.0.0.61)
Requirement already satisfied: nvidia-cufile==1.15.1.6.* in /home/msoliviawa
tt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit[c
ublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvt
x]==13.0.2; platform_system == "Linux"->torch) (1.15.1.6)
Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in /home/msolivi
awatt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolki
t[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,
nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.85)
Requirement already satisfied: nvidia-curand==10.4.0.35.* in /home/msoliviaw
att/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit
[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,n
vtx]==13.0.2; platform_system == "Linux"->torch) (10.4.0.35)
Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in /home/msolivi
awatt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolki
t[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,
nvtx]==13.0.2; platform_system == "Linux"->torch) (12.0.4.66)
Requirement already satisfied: nvidia-cusparse==12.6.3.3.* in /home/msolivia
watt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit
[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,n
vtx]==13.0.2; platform_system == "Linux"->torch) (12.6.3.3)
Requirement already satisfied: nvidia-nvjitlink==13.0.88.* in /home/msolivia
watt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit
[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,n
vtx]==13.0.2; platform_system == "Linux"->torch) (13.0.88)
Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in /home/msolivi
awatt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolki
```

```
t[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,
nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.88)
Requirement already satisfied: nvidia-nvtx==13.0.85.* in /home/msoliviawatt/
machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit[cubl
as,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]=
=13.0.2; platform_system == "Linux"->torch) (13.0.85)
Requirement already satisfied: cuda-pathfinder~=1.1 in /home/msoliviawatt/ma
chine_learning/ml-env/lib/python3.13/site-packages (from cuda-bindings<14,>=
13.0.3->torch) (1.5.1)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /home/msoliviawatt/m
achine_learning/ml-env/lib/python3.13/site-packages (from torchvision) (12.
1.1)
Requirement already satisfied: scipy-openblas32>=0.3.26 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from pennylane-light
ning>=0.44->pennylane) (0.3.31.188.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from sympy>=1.13.3->torch)
(1.3.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from astunparse->diastatic
-malt==2.15.2->pennylane) (0.46.3)
Requirement already satisfied: six<2.0,>=1.6.1 in /home/msoliviawatt/machine
_learning/ml-env/lib/python3.13/site-packages (from astunparse->diastatic-ma
lt==2.15.2->pennylane) (1.17.0)
Requirement already satisfied: MarkupSafe>=2.0 in /home/msoliviawatt/machine
_learning/ml-env/lib/python3.13/site-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from requests->penny
lane) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /home/msoliviawatt/machine_le
arning/ml-env/lib/python3.13/site-packages (from requests->pennylane) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from requests->pennylane)
(2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from requests->pennylane)
(2026.1.4)
Note: you may need to restart the kernel to use updated packages.
```

Quanvolutional Neural Networks {#quanvolution}

> > Warning
>
> This demo uses TensorFlow, which is a deprecated interface with PennyLane v0.42. Interfacing with TensorFlow will no longer be supported with PennyLane v0.43 and higher. Consider switching to a different machine learning interface with PennyLane, like PyTorch or JAX.

In this demo we implement the *Quanvolutional Neural Network*, a quantum machine learning model originally introduced in Henderson et al. (2019).



Introduction

Classical convolution

The *convolutional neural network* (CNN) is a standard model in classical machine learning which is particularly suitable for processing images. The model is based on the idea of a *convolution layer* where, instead of processing the full input data with a global function, a local convolution is applied.

If the input is an image, small local regions are sequentially processed with the same kernel. The results obtained for each region are usually associated to different channels of a single output pixel. The union of all the output pixels produces a new image-like object, which can be further processed by additional layers.

Quantum convolution

One can extend the same idea also to the context of quantum variational circuits. A possible approach is given by the following procedure which is very similar to the one used in Ref. [1]. The scheme is also represented in the figure at the top of this tutorial.

1. A small region of the input image, in our example a 2×2 square, is embedded into a quantum circuit. In this demo, this is achieved with parametrized rotations applied to the qubits initialized in the ground state.
2. A quantum computation, associated to a unitary U, is performed on the system. The unitary could be generated by a variational quantum circuit or, more simply, by a random circuit as proposed in Ref. [1].
3. The quantum system is finally measured, obtaining a list of classical expectation values. The measurement results could also be classically post-processed as proposed in Ref. [1] but, for simplicity, in this demo we directly use the raw expectation values.
4. Analogously to a classical convolution layer, each expectation value is mapped to a different channel of a single output pixel.
5. Iterating the same procedure over different regions, one can scan the full input image, producing an output object which will be structured as a multi-channel image.
6. The quantum convolution can be followed by further quantum layers or by classical layers.

The main difference with respect to a classical convolution is that a quantum circuit can generate highly complex kernels whose computation could be, at least in principle, classically intractable.

General setup

This Python code requires *PennyLane* with the *TensorFlow* interface and the plotting library *matplotlib*.

In [212…

```
import pennylane as qml
from pennylane import numpy as np
from pennylane.templates import RandomLayers
import matplotlib.pyplot as plt

# removed tensorflow and added pytorch stuff
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

import os
```

Setting of the main hyper-parameters of the model

In [213…

```
# macros
NUM_EPOCHS = 50
```

```
NUM_LAYERS = 1
NUM_TRAIN = 200
NUM_TEST = 80
NUM_CLASSES = 10
LEARNING_RATE = 0.001
BATCH_SIZE = 32
SQUARE_SIZE = 32

# change this to whatever path of data you want to use
SAVE_PATH = "./quanv_data/dataset_32_augmented/"

os.makedirs(SAVE_PATH, exist_ok=True)

# set this to true if your system does not have the preprocessed dataset dow
PREPROCESS = True

np.random.seed(0)
# tf.random.set_seed(0)
```

Loading of the Stanford Cars dataset

In [214…

```
from torchvision.datasets import StanfordCars
from torchvision import transforms
from torch.utils.data import Dataset, Subset, random_split, DataLoader
import torch

test_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((SQUARE_SIZE, SQUARE_SIZE)),
    transforms.ToTensor()
])

train_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((SQUARE_SIZE, SQUARE_SIZE)),

    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.1, contrast=0.1),
    transforms.GaussianBlur(kernel_size=3, sigma=(0.1, 1.0)),

    transforms.ToTensor()
])

full_data_train = StanfordCars(
    root="./data",
    split="train",
    download=False,
    transform=train_transform
)

full_data_test = StanfordCars(
    root="./data",
    split="train",
```

```
    download=False,
    transform=test_transform
)

print(len(full_data_train.classes))
```

196

In [215…

```
for i, name in enumerate(full_data_train.classes[:NUM_CLASSES]):
    print(i, name)
```

```
0 AM General Hummer SUV 2000
1 Acura RL Sedan 2012
2 Acura TL Sedan 2012
3 Acura TL Type-S 2008
4 Acura TSX Sedan 2012
5 Acura Integra Type R 2001
6 Acura ZDX Hatchback 2012
7 Aston Martin V8 Vantage Convertible 2012
8 Aston Martin V8 Vantage Coupe 2012
9 Aston Martin Virage Convertible 2012
```

In [216…

```
selected_classes = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
NUM_CLASSES = len(selected_classes)
print(NUM_CLASSES)
```

10

In [217…

```
# manually splitting the data
selected_indices = [
    i for i, (_, label) in enumerate(full_data_test._samples)
    if label in selected_classes
]

filtered_train_data = Subset(full_data_train, selected_indices)
filtered_test_data = Subset(full_data_test, selected_indices)

class_map = {old_label: new_label for new_label, old_label in enumerate(sele

class RemappedSubset(Dataset):
    def __init__(self, subset, class_map):
        self.subset = subset
        self.class_map = class_map

    def __len__(self):
        return len(self.subset)

    def __getitem__(self, idx):
        image, label = self.subset[idx]
        return image, self.class_map[label]

remapped_train_data = RemappedSubset(filtered_train_data, class_map)
remapped_test_data = RemappedSubset(filtered_test_data, class_map)

total_size = len(remapped_train_data)
train_size = int(0.8 * total_size)
val_size = total_size - train_size
```

```
generator = torch.Generator().manual_seed(42)

train_indices, val_indices = random_split(
    range(total_size),
    [train_size, val_size],
    generator=generator
)

train_subset = Subset(remapped_train_data, train_indices.indices)
val_subset = Subset(remapped_test_data, val_indices.indices)

# train_dl = DataLoader(train_subset, batch_size=16, shuffle=True)
# val_dl = DataLoader(val_subset, batch_size=16, shuffle=False)

# print("Total filtered samples:", len(remapped_data))
# print("Train samples:", len(train_subset))
# print("Val samples:", len(val_subset))
# print("Class map:", class_map)
```

Quantum circuit as a convolution kernel

We follow the scheme described in the introduction and represented in the figure at the top of this demo.

We initialize a PennyLane `default.qubit` device, simulating a system of 4 qubits. The associated `qnode` represents the quantum circuit consisting of:

1. an embedding layer of local R_y rotations (with angles scaled by a factor of π);
2. a random circuit of `NUM_LAYERS` ;
3. a final measurement in the computational basis, estimating 4 expectation values.

In [218…

```
dev = qml.device("default.qubit", wires=4)
# Random circuit parameters
rand_params = np.random.uniform(high=2 * np.pi, size=(NUM_LAYERS, 4))

@qml.qnode(dev)
def circuit(phi):
    # Encoding of 4 classical input values
    for j in range(4):
        qml.RY(np.pi * phi[j], wires=j)

    # Random quantum circuit
    RandomLayers(rand_params, wires=list(range(4)))

    # Measurement producing 4 classical output values
    return [qml.expval(qml.PauliZ(j)) for j in range(4)]
```

The next function defines the convolution scheme:

1. the image is divided into squares of 2×2 pixels;
2. each square is processed by the quantum circuit;
3. the 4 expectation values are mapped into 4 different channels of a single output pixel.

> > Note
>
> This process halves the resolution of the input image. In the standard language of CNN, this would correspond to a convolution with a 2×2 *kernel* and a *stride* equal to 2.

In [219…

```
def quanv(image):
    h, w, c = image.shape
    out = np.zeros((h // 2, w // 2, 4))

    for j in range(0, h - 1, 2):
        for k in range(0, w - 1, 2):
            q_results = circuit([
                image[j, k, 0],
                image[j, k + 1, 0],
                image[j + 1, k, 0],
                image[j + 1, k + 1, 0]
            ])

            for ch in range(4):
                out[j // 2, k // 2, ch] = q_results[ch]

    return out
```

In [220…

```
# NUM_TRAIN = 200
# NUM_TEST = 80
train_subset_small = Subset(train_subset, range(min(NUM_TRAIN, len(train_sub
val_subset_small = Subset(val_subset, range(min(NUM_TEST, len(val_subset))))

def dataset_to_numpy(dataset):
    images = []
    labels = []

    for img, label in dataset:
        # img is [C, H, W], but PennyLane tutorial expects [H, W, C]
        img_np = img.permute(1, 2, 0).numpy()
        images.append(img_np)
        labels.append(label)

    images = np.array(images)
    labels = np.array(labels)

    return images, labels

train_images, train_labels = dataset_to_numpy(train_subset_small)
```

```
test_images, test_labels = dataset_to_numpy(val_subset_small)

print("train_images shape: ", train_images.shape)
print("train_labels shape: ", train_labels.shape)
print("test_images shape: ", test_images.shape)
print("test_labels shape: ", test_labels.shape)
print("Unique train labels: ", np.unique(train_labels))
print("Unique test labels: ", np.unique(test_labels))
```

```
train_images shape:  (200, 32, 32, 1)
train_labels shape:  (200,)
test_images shape:  (80, 32, 32, 1)
test_labels shape:  (80,)
Unique train labels:  [0 1 2 3 4 5 6 7 8 9]
Unique test labels:  [0 1 2 3 4 5 6 7 8 9]
```

Quantum pre-processing of the dataset

Since we are not going to train the quantum convolution layer, it is more efficient to apply it as a "pre-processing" layer to all the images of our dataset. Later an entirely classical model will be directly trained and tested on the pre-processed dataset, avoiding unnecessary repetitions of quantum computations.

The pre-processed images will be saved in the folder `SAVE_PATH`. Once saved, they can be directly loaded by setting `PREPROCESS = False`, otherwise the quantum convolution is evaluated at each run of the code.

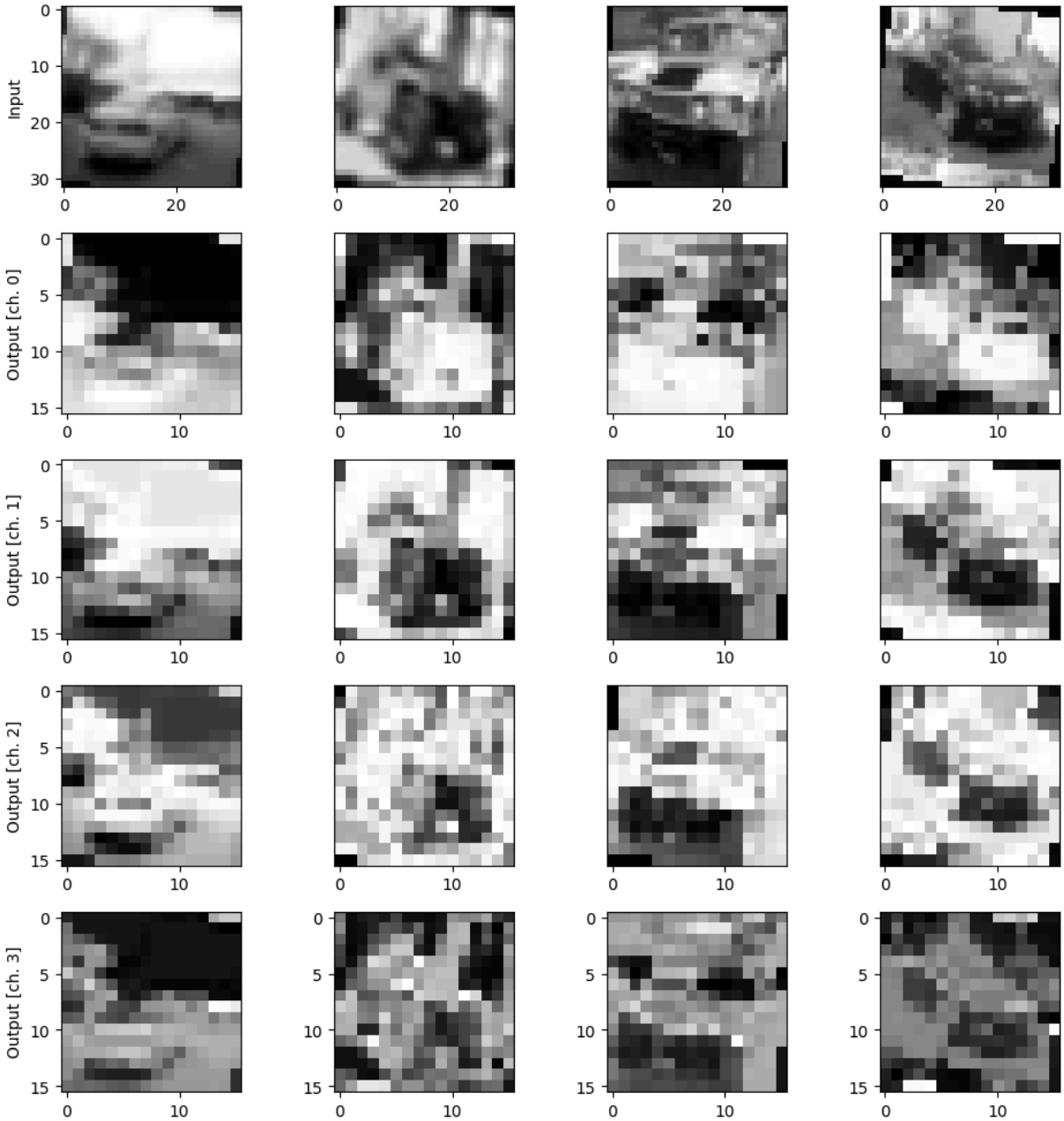
In [ ]:
```
if PREPROCESS == True:
    q_train_images = []
    print("Quantum pre-processing of train images:")
    for idx, img in enumerate(train_images):
        print("{}/{}        ".format(idx + 1, NUM_TRAIN), end="\r")
        q_train_images.append(quanv(img))
    q_train_images = np.asarray(q_train_images)

    q_test_images = []
    print("\nQuantum pre-processing of test images:")
    for idx, img in enumerate(test_images):
        print("{}/{}        ".format(idx + 1, NUM_TEST), end="\r")
        q_test_images.append(quanv(img))
    q_test_images = np.asarray(q_test_images)

    # Save pre-processed images
    np.save(SAVE_PATH + "q_train_images.npy", q_train_images)
    np.save(SAVE_PATH + "q_test_images.npy", q_test_images)


# Load pre-processed images
q_train_images = np.load(SAVE_PATH + "q_train_images.npy")
q_test_images = np.load(SAVE_PATH + "q_test_images.npy")
```

```
Quantum pre-processing of train images:
107/200
```

In [ ]:
```
q_train_images = torch.tensor(q_train_images, dtype=torch.float32)
q_test_images = torch.tensor(q_test_images, dtype=torch.float32)

train_images = torch.tensor(train_images, dtype=torch.float32)
test_images = torch.tensor(test_images, dtype=torch.float32)

train_labels = torch.tensor(train_labels, dtype=torch.long)
test_labels = torch.tensor(test_labels, dtype=torch.long)
```

Let us visualize the effect of the quantum convolution layer on a batch of samples:

In [ ]:
```
n_samples = 4
n_channels = 4
fig, axes = plt.subplots(1 + n_channels, n_samples, figsize=(10, 10))
for k in range(n_samples):
    axes[0, 0].set_ylabel("Input")
    if k != 0:
        axes[0, k].yaxis.set_visible(False)
    axes[0, k].imshow(train_images[k, :, :, 0], cmap="gray")

    # Plot all output channels
    for c in range(n_channels):
        axes[c + 1, 0].set_ylabel("Output [ch. {}]".format(c))
        if k != 0:
            axes[c, k].yaxis.set_visible(False)
        axes[c + 1, k].imshow(q_train_images[k, :, :, c], cmap="gray")

plt.tight_layout()
plt.show()
```



Below each input image, the 4 output channels generated by the quantum convolution are visualized in gray scale.

One can clearly notice the downsampling of the resolution and some local distortion introduced by the quantum kernel. On the other hand the global shape of the image is preserved, as expected for a convolution layer.

Hybrid quantum-classical model

After the application of the quantum convolution layer we feed the resulting features into a classical neural network that will be trained to classify the 10 different digits of the Stanford cars dataset.

We use a very simple model: just a fully connected layer with 10 output nodes with a final *softmax* activation function.

The model is compiled with a *stochastic-gradient-descent* optimizer, and a *cross-entropy* loss function.

In [ ]:
```
# def MyModel():
#     """Initializes and returns a custom Keras model
#     which is ready to be trained."""
#     model = keras.models.Sequential([
#         keras.layers.Flatten(),
#         keras.layers.Dense(num_classes, activation="softmax")
#     ])

#     model.compile(
#         optimizer='adam',
#         loss="sparse_categorical_crossentropy",
#         metrics=["accuracy"],
#     )
#     return model

class MyModel(nn.Module):
    def __init__(self, input_shape, num_classes=10):
        super().__init__()
        self.flatten = nn.Flatten()
        self.net = nn.Sequential(
            nn.Linear(input_shape, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.flatten(x)
        return self.net(x)
```

Training

We first initialize an instance of the model, then we train and validate it with the dataset that has been already pre-processed by a quantum convolution.

In [ ]:
```
def evaluate(model, images, labels):
    model.eval()
    with torch.no_grad():
        outputs = model(images)
        preds = torch.argmax(outputs, dim=1)
        acc = (preds == labels).float().mean().item()
    return acc
```

In [ ]:
```
def train_model(model, train_images, train_labels,
                val_images, val_labels, epochs):
```

```
    optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
    loss_fn = torch.nn.CrossEntropyLoss()

    train_history = []
    val_history = []

    for epoch in range(epochs):
        model.train()

        outputs = model(train_images)
        loss = loss_fn(outputs, train_labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # training accuracy
        train_preds = torch.argmax(outputs, dim=1)
        train_acc = (train_preds == train_labels).float().mean().item()

        # validation accuracy
        val_acc = evaluate(model, val_images, val_labels)

        train_history.append(train_acc)
        val_history.append(val_acc)

        print(f"Epoch {epoch+1}: train={train_acc:.4f}, val={val_acc:.4f}")

    return train_history, val_history
```

In [ ]:

```
input_size = q_train_images.shape[1] * q_train_images.shape[2] * q_train_ima

q_model = MyModel(input_size)

# q_history = train_model(q_model, q_train_images, train_labels, epochs = NU

q_train_hist, q_val_hist = train_model(
    q_model,
    q_train_images, train_labels,
    q_test_images, test_labels,
    epochs=NUM_EPOCHS
)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[199], line 7
      3 q_model = MyModel(input_size)
      5 # q_history = train_model(q_model, q_train_images, train_labels, epo
chs = NUM_EPOCHS)
----> 7 q_train_hist, q_val_hist = train_model(
      8     q_model,
      9     q_train_images, train_labels,
     10     q_test_images, test_labels,
     11     epochs=NUM_EPOCHS
     12 )

Cell In[198], line 13, in train_model(model, train_images, train_labels, val
_images, val_labels, epochs)
     10 for epoch in range(epochs):
     11     model.train()
---> 13     outputs = model(train_images)
     14     loss = loss_fn(outputs, train_labels)
     16     optimizer.zero_grad()

File ~/machine_learning/ml-env/lib/python3.13/site-packages/torch/nn/module
s/module.py:1779, in Module._wrapped_call_impl(self, *args, **kwargs)
   1777     return self._compiled_call_impl(*args, **kwargs)  # type: ignore
[misc]
   1778 else:
-> 1779     return self._call_impl(*args, **kwargs)

File ~/machine_learning/ml-env/lib/python3.13/site-packages/torch/nn/module
s/module.py:1790, in Module._call_impl(self, *args, **kwargs)
   1785 # If we don't have any hooks, we want to skip the rest of the logic
in
   1786 # this function, and just call forward.
   1787 if not (self._backward_hooks or self._backward_pre_hooks or self._fo
rward_hooks or self._forward_pre_hooks
   1788         or _global_backward_pre_hooks or _global_backward_hooks
   1789         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1790     return forward_call(*args, **kwargs)
   1792 result = None
   1793 called_always_called_hooks = set()

Cell In[196], line 27, in MyModel.forward(self, x)
     26 def forward(self, x):
---> 27     x = self.flatten(x)
     28     return self.net(x)

File ~/machine_learning/ml-env/lib/python3.13/site-packages/torch/nn/module
s/module.py:1779, in Module._wrapped_call_impl(self, *args, **kwargs)
   1777     return self._compiled_call_impl(*args, **kwargs)  # type: ignore
[misc]
   1778 else:
-> 1779     return self._call_impl(*args, **kwargs)

File ~/machine_learning/ml-env/lib/python3.13/site-packages/torch/nn/module
s/module.py:1790, in Module._call_impl(self, *args, **kwargs)
   1785 # If we don't have any hooks, we want to skip the rest of the logic
```

```
in
   1786 # this function, and just call forward.
   1787 if not (self._backward_hooks or self._backward_pre_hooks or self._fo
rward_hooks or self._forward_pre_hooks
   1788         or _global_backward_pre_hooks or _global_backward_hooks
   1789         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1790     return forward_call(*args, **kwargs)
   1792 result = None
   1793 called_always_called_hooks = set()

File ~/machine_learning/ml-env/lib/python3.13/site-packages/torch/nn/module
s/flatten.py:55, in Flatten.forward(self, input)
     51 def forward(self, input: Tensor) -> Tensor:
     52     """
     53     Runs the forward pass.
     54     """
---> 55     return input.flatten(self.start_dim, self.end_dim)

TypeError: flatten() takes from 0 to 1 positional arguments but 2 were given
```

In order to compare the results achievable with and without the quantum convolution layer, we initialize also a "classical" instance of the model that will be directly trained and validated with the raw car images (i.e., without quantum pre-processing).

In [ ]:
```
input_size = train_images.shape[1] * train_images.shape[2] * train_images.sh

c_model = MyModel(input_size)

# c_history = train_model(c_model, train_images, train_labels, epochs = NUM_
c_train_hist, c_val_hist = train_model(
    c_model,
    train_images, train_labels,
    test_images, test_labels,
    epochs=NUM_EPOCHS
)
```

```
Epoch 1: Loss=17.2034, Acc=0.1050
Epoch 2: Loss=15.7266, Acc=0.1400
Epoch 3: Loss=14.8563, Acc=0.2000
Epoch 4: Loss=14.2858, Acc=0.2700
Epoch 5: Loss=14.2019, Acc=0.2400
Epoch 6: Loss=13.7476, Acc=0.3000
Epoch 7: Loss=12.9194, Acc=0.3650
Epoch 8: Loss=12.7687, Acc=0.3500
Epoch 9: Loss=12.7461, Acc=0.3900
Epoch 10: Loss=12.4103, Acc=0.3600
Epoch 11: Loss=11.7876, Acc=0.4750
Epoch 12: Loss=11.5938, Acc=0.4400
Epoch 13: Loss=11.0427, Acc=0.5050
Epoch 14: Loss=10.9799, Acc=0.5000
Epoch 15: Loss=10.9052, Acc=0.5950
Epoch 16: Loss=10.4324, Acc=0.5250
Epoch 17: Loss=9.7907, Acc=0.5950
Epoch 18: Loss=9.3931, Acc=0.6000
Epoch 19: Loss=9.3682, Acc=0.6050
Epoch 20: Loss=8.8417, Acc=0.6400
Epoch 21: Loss=8.6139, Acc=0.6750
Epoch 22: Loss=8.2695, Acc=0.6050
Epoch 23: Loss=9.0326, Acc=0.6700
Epoch 24: Loss=8.4213, Acc=0.6000
Epoch 25: Loss=8.5753, Acc=0.6350
Epoch 26: Loss=8.0480, Acc=0.6500
Epoch 27: Loss=7.4473, Acc=0.7100
Epoch 28: Loss=7.3530, Acc=0.6750
Epoch 29: Loss=6.9380, Acc=0.7600
Epoch 30: Loss=6.7988, Acc=0.7700
Epoch 31: Loss=6.4449, Acc=0.7600
Epoch 32: Loss=6.3316, Acc=0.7900
Epoch 33: Loss=6.0815, Acc=0.7800
Epoch 34: Loss=5.5797, Acc=0.8150
Epoch 35: Loss=5.5514, Acc=0.8400
Epoch 36: Loss=5.1059, Acc=0.8500
Epoch 37: Loss=5.3563, Acc=0.8350
Epoch 38: Loss=5.1566, Acc=0.8600
Epoch 39: Loss=4.7593, Acc=0.8600
Epoch 40: Loss=4.6808, Acc=0.8950
Epoch 41: Loss=4.6162, Acc=0.8800
Epoch 42: Loss=4.8404, Acc=0.8650
Epoch 43: Loss=4.3253, Acc=0.8900
Epoch 44: Loss=3.9565, Acc=0.9000
Epoch 45: Loss=3.9254, Acc=0.9050
Epoch 46: Loss=3.8848, Acc=0.9300
Epoch 47: Loss=3.4141, Acc=0.9150
Epoch 48: Loss=3.6968, Acc=0.9450
Epoch 49: Loss=3.2789, Acc=0.9100
Epoch 50: Loss=3.2302, Acc=0.9450
```

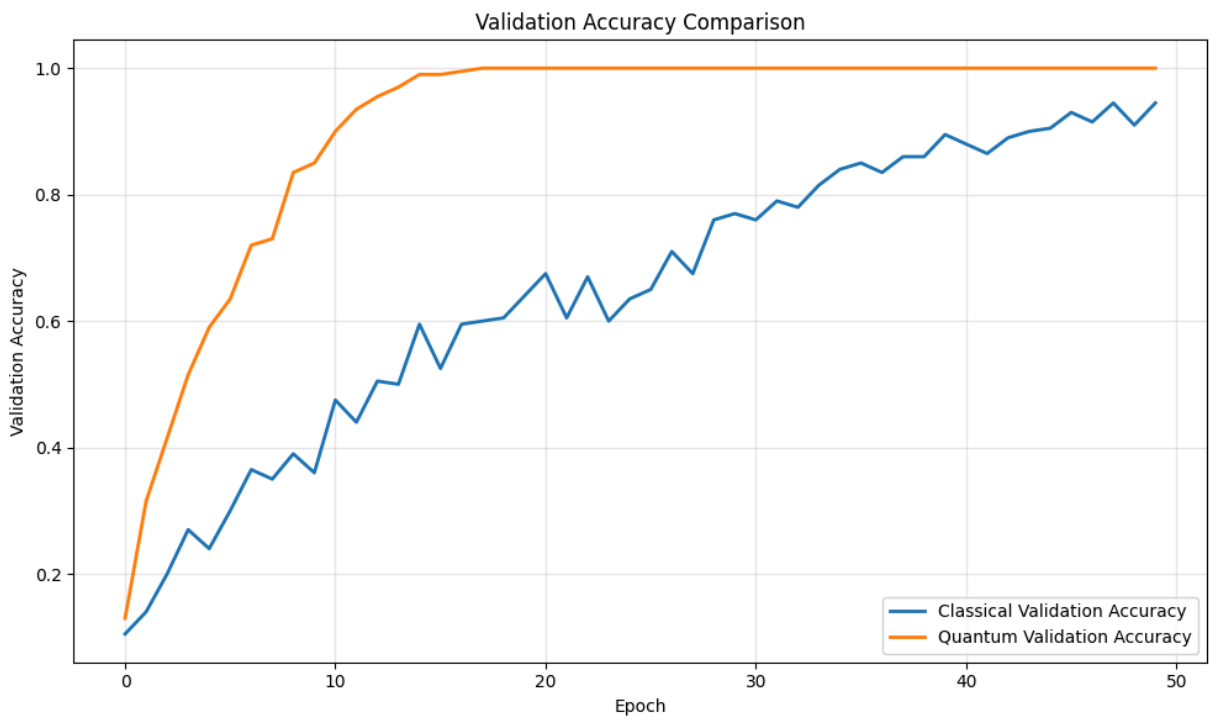
In [ ]:

```python
import matplotlib.pyplot as plt

plt.plot(q_train_hist, label="Quantum Train Accuracy")
plt.plot(q_val_hist, label="Quantum Validation Accuracy")
plt.plot(c_train_hist, label="Classical Train Accuracy")
```

```
plt.plot(c_val_hist, label="Classical Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Training vs Validation Accuracy")

plt.show()
```

In [ ]:

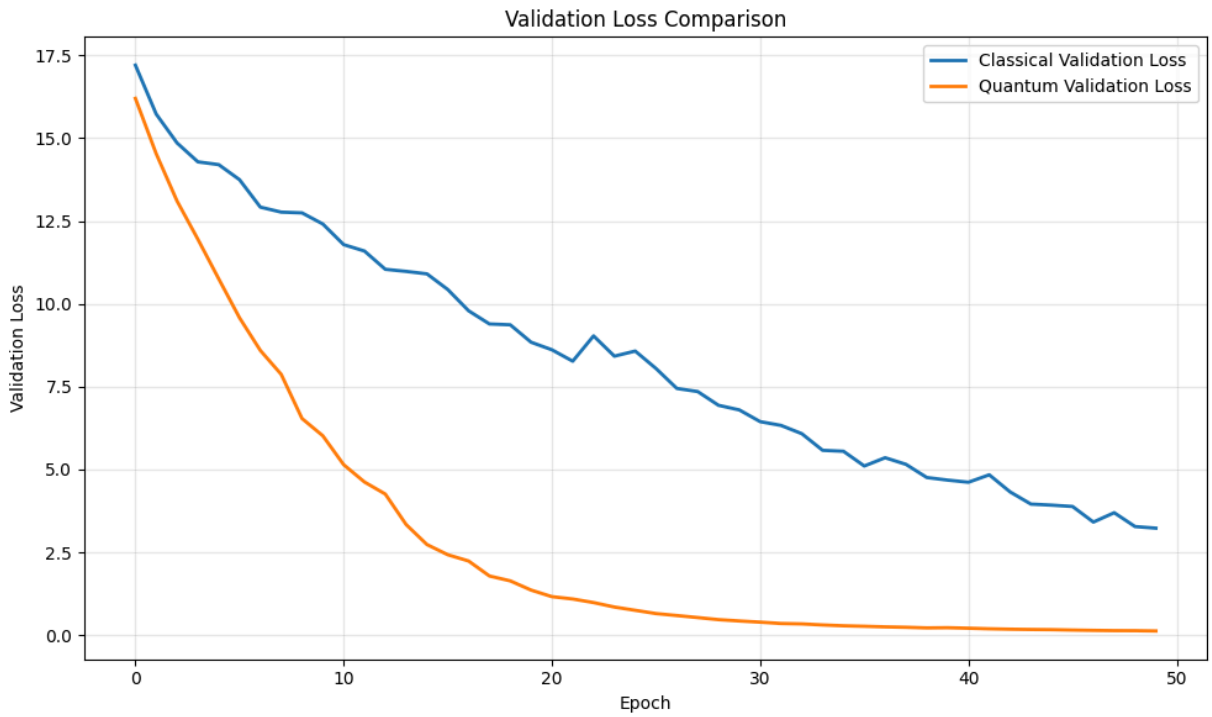
```
# plt.figure(figsize=(10, 6))
# plt.plot(c_history["accuracy"], label="Classical Train Accuracy", linewidt
# plt.plot(q_history["accuracy"], label="Quantum Train Accuracy", linewidth=

# plt.title("Train Accuracy Comparison")
# plt.xlabel("Epoch")
# plt.ylabel("Train Accuracy")
# plt.legend()
# plt.grid(True, alpha=0.3)
# plt.tight_layout()
# plt.show()

# # plt.plot(q_history["accuracy"], label="Quantum")
# # plt.plot(c_history["accuracy"], label="Classical")
# # plt.legend()
```



Results

We can finally plot the test accuracy and the test loss with respect to the number of training epochs.

In [ ]:
```
# plt.figure(figsize=(10, 6))
# plt.plot(c_history["loss"], label="Classical Train Loss", linewidth=2)
# plt.plot(q_history["loss"], label="Quantum Train Loss", linewidth=2)

# plt.title("Train Loss Comparison")
# plt.xlabel("Epoch")
# plt.ylabel("Train Loss")
# plt.legend()
# plt.grid(True, alpha=0.3)
# plt.tight_layout()
# plt.show()
```



1. Maxwell Henderson, Samriddhi Shakya, Shashindra Pradhan, Tristan Cook. "Quanvolutional Neural Networks: Powering Image Recognition with Quantum Circuits." arXiv:1904.04767, 2019.